Departamento de Computación
FCEFQyN, Universidad Nacional de Río Cuarto
Asignatura: Análisis y Diseño de Algoritmos I
Segundo Cuatrimestre de 2025

Guía Práctica No. 1: Complejidad Temporal de Algoritmos: Análisis de Tiempo de Ejecución

Esta guía práctica abarca, análisis de tiempo de ejecución en el peor caso, mejor caso y caso promedio. Se recomienda fuertemente la lectura del capítulo 2 de *Introduction to the Design and Analysis of Algorithms* (Levitin 2003) y las secciones C.1, C.2 y C.3 de *Introduction to Algorithms* (Thomas H.Cormen)

Parte de esta práctica cuenta con esquemas de programación y tests de asistencia a su resolución. Éstos pueden accederse a partir del classroom de github, para el cual es conveniente tener una cuenta en Github (`http://github.com`).

Esta práctica no tiene entrega formal. Su resolución es opcional aunque se recomienda realizar cada uno de los ejercicios propuestos.

Primera Parte

1. Considere la clase `ArraySorter` provista en el repositorio. Implemente los algoritmos de ordenamiento `selectionSort` y `bubbleSort`. Asegúrese que todos los tests pasen (debe completar la implementación de algunos métodos auxiliares). Complete el conjunto de tests de unidad provistos, con otros adicionales, para los algoritmos implementados. Utilice el proyecto para realizar una comparación experimental del tiempo de ejecución de los algoritmos implementados.

2. Dado el siguiente algoritmo para BUBBLE-SORT: Muestre que el algoritmo corre en $O(n^2)$

```
BUBLESORT (A)
1  for i = 1 to A.length - 1
2    for j = A.length downto i + 1
3      if A[j] < A[j-1]
4        swap A[j] and A[j-1]
```

3. Realizar el análisis de tiempo de ejecución de MergeSort.

4. Analizar el tiempo de ejecución del método *hasDuplicates*(*int*[ ] *arr*) en el peor caso, aclarar qué considera como el tamaño del input, y cuál sería el peor caso, desarrolle la ecuación del tiempo de ejecución y determine la tasa de crecimiento.

```
/**
 * Checks if there are duplicates in the array.
 */
public static boolean hasDuplicates(int[] arr) {
    return hasDuplicates(arr, 0);
}

/**
 * Recursively checks if there are duplicated elements in the array.
 */
```

```java
public static boolean hasDuplicates(int[] arr, int index) {
    if (index >= arr.length) {
        return false;
    }
    if (sequentialSearch(arr, arr[index], index + 1) != -1) {
        return true;
    }
    return hasDuplicates(arr, index + 1);
}

/**
 * Recursively searches for an element in the array.
 */
public static int sequentialSearch(int[] arr, int target, int index) {
    if (index >= arr.length) {
        return -1;
    }
    if (arr[index] == target) {
        return index;
    }
    return sequentialSearch(arr, target, index + 1);
}
```

Segunda Parte

1. Como hemos visto en la clase teórica, el número promedio de comparaciones de claves realizadas por la búsqueda secuencial está dado por la fórmula:

$$C_{\text{avg}}(n) = \frac{p(n+1)}{2} + n(1-p)$$

   donde p es la probabilidad de una búsqueda exitosa. Determine, para un n fijo, los valores de p $(0 \leq p \leq 1)$ para los cuales esta fórmula produce el valor máximo de $C_{\text{avg}}(n)$ y el valor mínimo de $C_{\text{avg}}(n)$.

2. Imagina que después de lavar 5 pares distintos de calcetines, descubres que faltan dos calcetines. Por supuesto, te gustaría conservar la mayor cantidad posible de pares completos.

   En el mejor de los casos, te quedarías con 4 pares completos, y en el peor de los casos, con 3 pares completos.

   Asumiendo que la probabilidad de desaparición de cada uno de los 10 calcetines es la misma, encuentra:

   - La probabilidad del mejor caso.
   - La probabilidad del peor caso.
   - El número esperado de pares completos en el caso promedio.

3. Se tiene el siguiente procedimiento de ordenamiento para insertion Sort:

```
void insertionSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        int key = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > key) {
```

```
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = key;
    }
}
```

- Estima el número total de comparaciones en el peor caso.
- Estima el número total de comparaciones en el mejor caso (cuando el arreglo ya está ordenado).
- Mostra que el número de comparaciones de claves en el caso promedio está dado por la fórmula:

$$C_{\text{avg}}(n) \approx \frac{n^2}{4}$$

4. Implemente el algoritmo Randomized-Quicksort tal como se presenta en el capítulo 7 del libro *Introduction to Algorithms (Thomas H. Cormen)*. Para seleccionar el pivote, utilice una función de generación de números aleatorios con distribución uniforme. Realice un estudio empírico del tiempo de ejecución del algoritmo sobre arreglos de tamaños crecientes, generados con números aleatorios distribuidos uniformemente. Grafique el tiempo medido en función del tamaño del arreglo. A partir de los resultados, analice y verifique experimentalmente que el tiempo promedio de ejecución del Randomized-Quicksort es $O(n\ log\ n)$